

Mardik v2 — Dossier de conception, Chantier 2 : l'observabilité qui pilote la chaîne

Livrable de conception (deuxième partie) — version de travail, 2026-09-16. Auteur : Eramalingam GANDAKUMAR (solo). Brief : Elbby Skermine AGWU, 14/09/2026. Périmètre : Chantier 2 seul — le schéma de la boucle d'observabilité et le tableau de pilotage ; il s'appuie sur le dossier du Chantier 1 (contrat /v2 , chaîne 11mops.yml , manifeste, entrées set_weight et redeploy) sans le modifier. Révisé le 18/09 après la séance de classe (section 13) ; le Chantier 1 n'est toujours pas validé. ⚠️ Rédigé **avant** la validation du Chantier 1 et **sans** le dépôt (ops/dashboard.py , deploy.py::surveiller , drift_proxy.py , le registre n'ont pas été lus) : ce qui en dépend porte la mention **[à confirmer]**.

Sommaire

1. Objectif et périmètre	2
2. Les cinq signaux	4
3. La dérive : référence, statistiques, seuils	4
4. Les trois boucles de rétroaction	5
4.1 Rollback sur signal — la décision est humaine	6
4.2 Promotion pilotée par les métriques — automatique, non compensable	6
4.3 Enrichissement du jeu d'évaluation — la production nourrit le gate suivant	6
5. Le journal de pilotage	7
6. Le watcher, l'alerte, le bouton et l'API de pilotage	7
6.1 Le watcher	7
6.2 L'alerte	8
6.3 Le bouton	8
6.4 Une API de pilotage, pas seulement un bouton	8
7. Traces et collecteur	8
8. Le tableau de bord	9
9. Ajuster les seuils	9
10. Schéma de la boucle d'observabilité	10
11. Tableau de pilotage	10
12. Démonstration et tests d'acceptance	11
12 bis. Choix d'outils non triviaux — ce qui est retenu, ce qui est écarté, pourquoi	12
13. Hypothèses, limites, à confirmer	12
14. Critères RNCP couverts	14
Annexe A — Réponses aux questions de cadrage (Chantier 2)	14
 Glossaire	15

1. Objectif et périmètre

Ce que le Chantier 2 fait. Le Chantier 1 rend la livraison d'une version automatique et réversible. Le Chantier 2 fait venir des **mesures** la décision d'avancer ou de reculer : la production est observée (cinq signaux sur un tableau de bord), des seuils transforment ces signaux en décisions, et trois rétroactions agissent sur la chaîne — un retour arrière sur signal, une promotion du canary pilotée par les métriques, un enrichissement du jeu d'évaluation par les cas réels. Chaque décision, humaine ou automatique, laisse une ligne dans un journal de pilotage.

Pour qui. Le chef de projet, qui regarde le tableau de bord et qui clique le retour arrière ; les juristes, dont les cas difficiles alimentent le jeu de référence ; le développeur, qui ajuste les seuils et lit les traces.

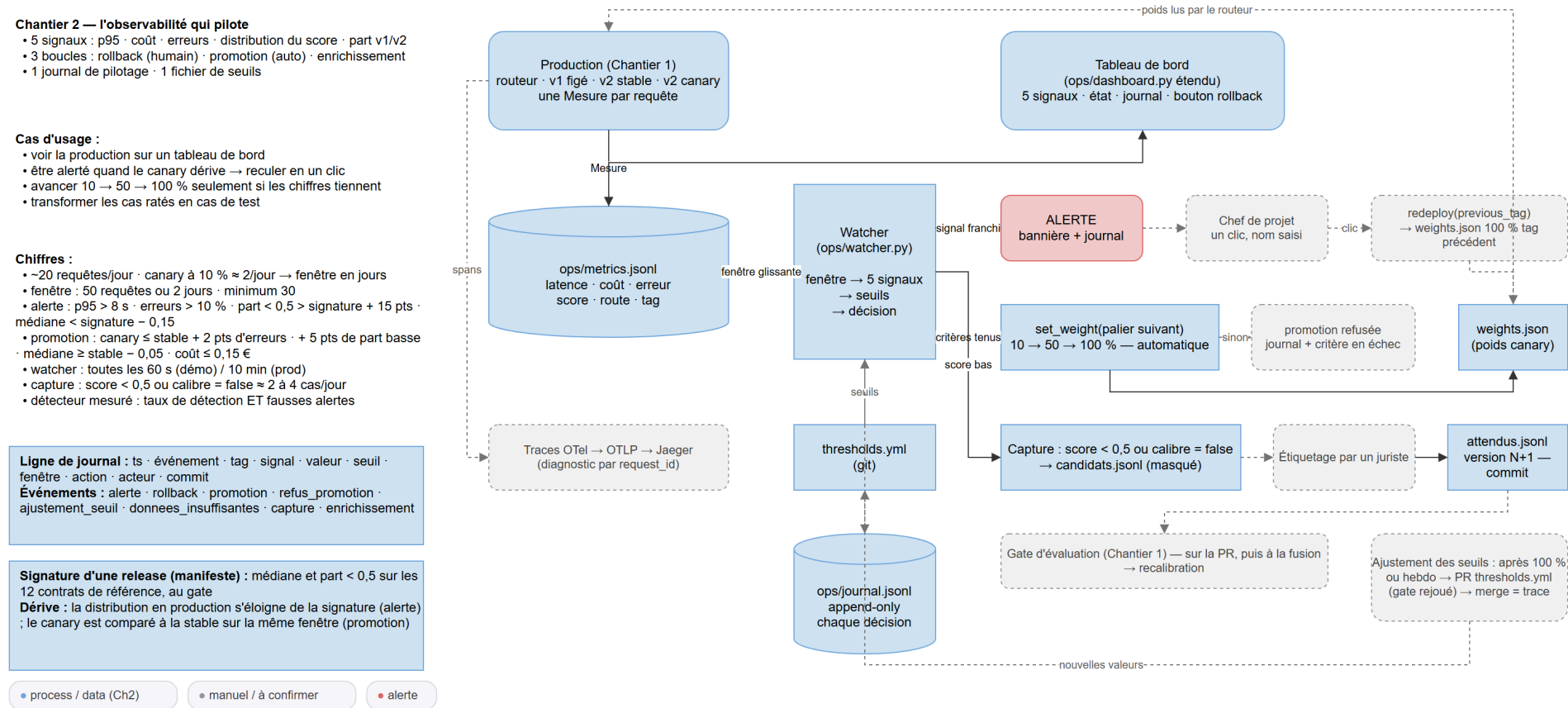
Les quatre points de raccord avec le Chantier 1 — les seules frontières, déjà dessinées sur le schéma de la chaîne :

Sens	Point de raccord	Qui l'actionne
depuis la production	les <code>Mesure</code> par requête dans <code>ops/metrics.jsonl</code>	lues par le watcher et le tableau de bord — lecture seule
vers la chaîne	<code>set_weight(tag, pct)</code>	le watcher, automatiquement, quand les critères de promotion tiennent
vers la chaîne	<code>redeploy(previous_tag)</code>	un humain, après une alerte — jamais le watcher
vers la chaîne	<code>attendus.jsonl</code> version N+1	la boucle d'enrichissement, par une pull request : le gate d'évaluation la rejoue dès la PR (filtre de chemin du Chantier 1, §3.0), puis à la fusion

Ce que le Chantier 2 ne touche pas. Le contrat `/v2`, la chaîne `11mops.yml`, le manifeste, le contrat `/v1`. Il ne fabrique aucun signal : il lit ceux que la production émet déjà.

Sur quoi il s'appuie. La télémétrie fournie par la remédiation (`app/telemetry.py` : `spans`, journaux structurés, `Mesure`), le tableau de bord et la surveillance fournis (`ops/dashboard.py`, `ops/deploy.py::surveiller`), le proxy de dérive (`ops/drift_proxy.py`) et le simulateur de trafic (`make traffic MODE=derive-score`). On construit dessus, on ne les refait pas — **[à confirmer]** : leur interface exacte n'a pas été lue.

Vue d'ensemble du Chantier 2. Avant le détail des signaux et des boucles, le chantier entier sur une page, à la manière d'un tableau blanc : à gauche ce qu'on mesure, les cas d'usage et les ordres de grandeur ; au centre le chemin d'une mesure, de la production au watcher puis aux trois rétroactions ; à droite ce que chaque boucle écrit.



Source éditable : `schemas/schema-boucle-tableau-blanc.drawio`.

2. Les cinq signaux

Question : quels signaux pilotent, et pourquoi pas la moyenne ? **Réponse courte :** cinq signaux tirés d'une seule ligne par requête ; pour le score, la part < 0,5 et la médiane, jamais la moyenne — elle cacherait un modèle sûr de lui et faux sur les contrats longs.

Tous viennent de la même source : une ligne de `ops/metrics.jsonl` par requête, écrite par v1 (déjà), par v2 et par le routeur (à ajouter, même format) — `ts`, `version`, `route`, `latence_ms`, `erreur`, `score`, `cout_eur`, `appels_llm`, `tokens`, `tronque`.

Signal	Calcul	Fenêtre	Pourquoi celui-là
Latence p95	95 ^e centile de <code>latence_ms</code> , par tag	50 requêtes ou 2 jours, minimum 30	la contrainte du client est un p95, pas une moyenne ; la moyenne cache les requêtes lentes
Coût	moyenne de <code>cout_eur</code> par analyse, par tag ; cumul du jour	idem ; cumul quotidien	contrainte du client (< 0,15 €) ; le cumul dit si le budget mensuel (≈ 60 €) tient
Taux d'erreur	part des lignes <code>erreur = true</code> , par tag	idem	signal le plus rapide d'un fournisseur ou d'un déploiement cassé
Distribution du score	part des scores < 0,5 (la bande « basse ») et médiane , par tag — jamais la moyenne	idem	la part < 0,5 est ce que le juriste ressent (combien de fois il relit) ; la médiane est le centre, robuste aux extrêmes ; une moyenne cacherait un modèle sûr de lui et faux sur les contrats longs, correct sur les courts — exactement le défaut à détecter
Part de trafic v1/v2	part des lignes par <code>route</code> , et par <code>tag</code> dans <code>/v2</code>	par jour	prouve « quelle version sert quel trafic » ; une part <code>/v2</code> éloignée du poids attendu signale un routage cassé

Un sixième état, pas un signal : **données insuffisantes** — moins de 30 requêtes dans la fenêtre. Il s'affiche comme tel, et aucune décision n'en sort. Un tableau de bord qui montre un p95 calculé sur quatre requêtes ment ; celui-ci dit « pas encore ».

Ce que les signaux ne sont pas : les traces OpenTelemetry servent au diagnostic d'une requête précise (section 7), pas au pilotage ; les journaux structurés racontent, ils ne décident pas. Piloter, c'est ce fichier et ces cinq calculs.

3. La dérive : référence, statistiques, seuils

Question : une dérive par rapport à quoi, et à partir de combien de requêtes ? **Réponse courte :** deux références — la signature de la release pour alerter, la version stable pour promouvoir —, 30 requêtes minimum, fenêtre de 50 requêtes ou 2 jours ; tous les seuils dans un seul fichier versionné.

Une dérive par rapport à quoi. Deux références, deux usages.

- **La signature de la release.** Au build, le gate d'évaluation calcule la distribution du score sur les 12 contrats de référence ; le manifeste en garde la médiane et la part < 0,5. Une version « dérive » quand sa distribution en production s'éloigne de ce qu'elle-même a obtenu en sortant de la chaîne. C'est la référence du **retour arrière** : mesurée par le gate, pas décidée.
- **Stable contre canary, même période.** Pour la **promotion**, on compare le canary à la version stable sur la même fenêtre. Si un lot de contrats difficiles arrive, les deux versions baissent ensemble, et le canary n'est pas accusé du trafic.

Quelles statistiques. La part des scores $< 0,5$ et la médiane (section 2) pour le score ; le taux d'erreur ; le p95 de latence. Le coût entre dans les critères de promotion, pas de retour arrière : une dérive de coût se corrige en jours, pas en minutes.

Combien de requêtes avant de parler de dérive. 30 au minimum dans la fenêtre — l'ordre de grandeur où une part et une médiane cessent de sauter d'une requête à l'autre ; en dessous, « données insuffisantes ». Fenêtre glissante : les 50 dernières requêtes ou les 2 derniers jours, la plus grande des deux. En production, ~20 requêtes par jour : le minimum est atteint en un jour et demi ; en démonstration, en quelques minutes avec le trafic simulé.

Les seuils, tous au même endroit — `thresholds.yml`, versionné en git ; en changer un est un commit :

```
version: 1
baseline: v2.0.0          # la release dont le manifeste sert de signature
fenetre:
  requetes_min: 30        # en dessous : données insuffisantes, aucune décision
  requetes: 50
  jours: 2
rollback_alerte:          # franchi → alerte ; la décision est humaine
  part_score_bas_delta_max: 0.15  # part  $< 0,5$  au-delà de la signature + 15 points
  mediane_delta_min: -0.15        # médiane sous la signature - 0,15
  taux_erreur_max: 0.10
  p95_ms_max: 8000
promotion:                # tous les critères, aucun ne compense un autre
  requetes_min: 30
  jours_min: 1
  p95_ms_max: 8000
  taux_erreur_delta_max: 0.02     # canary  $\leq$  stable + 2 points
  part_score_bas_delta_max: 0.05  # canary  $\leq$  stable + 5 points
  mediane_delta_min: -0.05        # canary  $\geq$  stable - 0,05
  cout_moyen_eur_max: 0.15
  paliers: [10, 50, 100]
  refus_consecutifs_alerte: 2
capture:
  score_global_max: 0.5
  calibre_false: true
```

Ces valeurs sont des **valeurs de travail** : les marges seront posées sur les premières distributions observées, puis ajustées selon la procédure de la section 9. Le principe, lui, ne bouge pas : mesuré, pas décidé « au doigt mouillé ».

Le détecteur se mesure dans les deux sens. Avant de lui confier une alerte : trafic dégradé par le proxy de dérive ($\text{DRIFT}=\text{score}$) → taux de détection ; trafic sain → taux de fausses alertes. Les deux chiffres figurent dans le dossier de développement. Un détecteur qui détecte bien mais crie trop souvent voit sa marge élargie, pas son alerte coupée.

Fail-closed. Fichier de métriques absent, illisible ou vide → état « données insuffisantes », aucune promotion, alerte technique. Une promotion ne part jamais sur une absence de données.

4. Les trois boucles de rétroaction

Question : qui décide d'avancer, qui décide de reculer ? **Réponse courte** : avancer est automatique quand les cinq critères tiennent tous ; reculer est un humain qui clique après une alerte — l'arbitrage du CTO ; et la production nourrit le gate suivant par une pull request.

L'asymétrie, posée une fois. Avancer est automatique : quand les chiffres tiennent, monter d'un palier est réversible et à faible regret. Reculer est une décision humaine : un incident se juge, il ne se seuille pas — c'est

l'arbitrage du CTO (un retour arrière automatique serait « antinomique » avec le canary), retenu tel quel. Le watcher détecte et alerte ; il ne recule jamais seul.

4.1 Rollback sur signal — la décision est humaine

1. Le watcher lit la fenêtre du tag qui sert du trafic ; un signal de `rollback_alerte` est franchi.
2. Il écrit une ligne de journal `alerte` (signal, valeur, seuil, fenêtre, tag) et met le tableau de bord en état d'alerte — bannière rouge.
3. Le chef de projet voit la bannière, saisit son nom, clique **Rollback**.
4. Le bouton appelle `redeploy(previous_tag)` — l'entrée de la chaîne : `weights.json` envoie 100 % de `/v2` vers le tag précédent, déjà validé par les gates ; rien n'est reconstruit.
5. Une ligne `rollback` est écrite : acteur = le nom saisi, cause = l'identifiant de l'alerte, tag quitté, tag repris.
6. Vérification après coup, écrite dans le runbook : le tableau de bord montre la part de trafic du tag précédent à 100 %, et les signaux reviennent dans leurs bornes sur la fenêtre suivante.

Ce qui bloque : le rollback n'est pas possible s'il n'existe aucun tag précédent validé — situation de la première release, dite explicitement.

4.2 Promotion pilotée par les métriques — automatique, non compensable

1. Le canary sert 10 %. Après **au moins 30 requêtes et au moins un jour** au palier courant, le watcher évalue les cinq critères de `promotion` : p95, taux d'erreur, part < 0,5, médiane — les trois derniers **relatifs à la version stable sur la même fenêtre** —, coût moyen.
2. **Tous tiennent** → `set_weight(tag, palier suivant)` ; ligne `promotion` (acteur = watcher, valeurs, palier) ; nouvelle fenêtre. 10 → 50 → 100.
3. **Un seul échoue** → aucune promotion ; ligne `refus_promotion` avec le critère en échec et sa valeur. Deux refus consécutifs → alerte pour décision humaine : attendre, ou reculer.
4. À 100 %, la version devient la nouvelle baseline (section 9).

Un bon score ne rachète jamais une mauvaise latence : les gates ne se compensent pas (même règle que le gate d'évaluation du Chantier 1). En démonstration, `requetes_min` et `jours_min` sont abaissés par configuration, jamais par le code.

4.3 Enrichissement du jeu d'évaluation — la production nourrit le gate suivant

1. **Capture.** Toute requête dont le score global est < 0,5 ou dont `calibre = false` est copiée dans `ops/candidats.jsonl` : texte, réponse du modèle, score, tag, `request_id`.
2. **Masquage.** Avant stockage, les noms des parties et les montants sont remplacés par regex (`PARTIE_A`, `PARTIE_B`, `MONTANT`). C'est partiel par construction — une regex ne voit pas tout — et la question de l'anonymisation reste **ouverte avec le client** ; le dossier le dit plutôt que de le supposer réglé.
3. **Étiquetage.** Un juriste écrit, pour chaque candidat, la liste des clauses attendues — le même format que `attendus.jsonl`. À ~20 requêtes par jour et une part basse de 10 à 20 %, cela fait deux à quatre cas par jour : étiquetable.
4. **Vernissage.** Un script ajoute les cas étiquetés à `attendus.jsonl` en version N+1 ; c'est un commit, avec son auteur.
5. **Rejeu.** Dès la pull request qui porte la version N+1 — le filtre de chemin du Chantier 1 (§3.0) déclenche le gate d'évaluation sur `attendus.jsonl` — puis à la fusion suivante, le gate rejoue ces cas, refait la calibration, et le manifeste de la release enregistre `golden_dataset_version: N+1` et `calibration_version` suivante.

C'est la boucle qui fait que le score reste calibré sur ce que la production apporte, au lieu de rester figé sur douze contrats.

5. Le journal de pilotage

Un fichier, `ops/journal.jsonl`, append-only — on ajoute, on ne réécrit jamais. Une ligne par événement :

Champ	Contenu
<code>ts</code>	horodatage ISO 8601, UTC
<code>evenement</code>	<code>alerte</code> · <code>rollback</code> · <code>promotion</code> · <code>refus_promotion</code> · <code>ajustement_seuil</code> · <code>donnees_insuffisantes</code> · <code>capture</code> · <code>enrichissement</code>
<code>tag</code>	la version concernée
<code>signal</code> , <code>valeur</code> , <code>seuil</code> , <code>fenetre</code>	ce qui a déclenché — le signal, sa valeur mesurée, le seuil comparé, la fenêtre (nombre de requêtes, bornes de temps)
<code>action</code>	ce qui a été fait (<code>set_weight 50</code> , <code>redeploy v2.0.0</code> , <code>thresholds.yml modifié</code> , ...) ou « aucune »
<code>acteur</code>	<code>watcher</code> , ou le nom saisi par l'humain
<code>commit</code>	pour un ajustement de seuil ou un enrichissement : le commit qui porte la modification
<code>commentaire</code>	libre, court

```
{ "ts": "2026-10-02T09:14:03Z", "evenement": "alerte", "tag": "v2.1.0", "signal": "part_score_bas", "valeur": 0.41, "seuil": 0.27, "fenetre": { "requetes": 50, "de": "2026-09-30T14:00Z", "a": "2026-10-02T09:10Z" }, "action": "aucune", "acteur": "watcher" }
{ "ts": "2026-10-02T09:21:40Z", "evenement": "rollback", "tag": "v2.1.0", "signal": "part_score_bas", "valeur": 0.41, "seuil": 0.27, "action": "redeploy v2.0.0 (100 %)", "acteur": "M. Dupont", "commentaire": "alerte 09:14" }
{ "ts": "2026-10-09T08:00:12Z", "evenement": "ajustement_seuil", "tag": "v2.1.1", "signal": "part_score_bas_delta_max", "valeur": 0.15, "seuil": 0.12, "action": "thresholds.yml modifié", "acteur": "E. Gandakumar", "commit": "a41c9e2", "commentaire": "distributions du 02 au 08/10" }
```

Le tableau de bord affiche les vingt dernières lignes ; le fichier entier est le livrable « journal de pilotage » (test d'acceptance 10). Il ne contient jamais de texte de contrat.

6. Le watcher, l'alerte, le bouton et l'API de pilotage

Question : que peut faire le watcher seul, et qu'est-ce qui l'en empêche ? **Réponse courte :** alerter et promouvoir ; jamais redéployer — et le rollback passe par une API documentée, pas seulement par un bouton.

6.1 Le watcher

`ops/watcher.py`, un processus Python dans son propre conteneur, une boucle toutes les 60 secondes en démonstration, toutes les 10 minutes en production. À chaque tour : lire la fenêtre de `metrics.jsonl` par tag (en réutilisant `deploy.py::surveiller` pour la lecture **[à confirmer]**), calculer les cinq signaux, comparer à `thresholds.yml`, écrire dans le journal, écrire l'état (`ops/etat.json` : par tag, `ok` · `alerte` · `donnees_insuffisantes`, avec la raison). Ce qu'il **peut** faire seul : alerter, promouvoir (`set_weight`). Ce qu'il **ne peut jamais** faire : `redeploy`. Pourquoi un processus à côté de la production et non une tâche planifiée du serveur d'intégration : un exécuteur distant ne voit pas une production locale ; le watcher vit là où sont les mesures.

6.2 L'alerte

Une bannière rouge sur le tableau de bord, une ligne de journal, et, si un canal existe, un message au chef de projet (courriel ; option, pas une exigence). L'alerte porte le signal, la valeur, le seuil et la fenêtre — de quoi décider sans ouvrir un fichier.

6.3 Le bouton

Sur le tableau de bord : « Rollback vers », un champ nom obligatoire, un clic. Un non-développeur peut le faire, comme le CTO l'a demandé.

6.4 Une API de pilotage, pas seulement un bouton

Le bouton et la bannière sont une interface ; ils ont besoin, derrière, d'un **contrat** — sinon le rollback n'existe que dans le code du tableau de bord, invérifiable et non rejouable par un script. `ops/pilotage.py` expose donc une petite API, avec la même discipline de contrat que `/v2` (section 2 du Chantier 1) : schémas figés, erreurs explicites au format RFC 9457, `request_id` sur chaque réponse.

Route	Rôle	Qui l'appelle
<code>GET /pilotage/etat</code>	l'état courant par tag : les cinq signaux, l'état (<code>ok</code> / <code>alerte</code> / <code>donnees_insuffisantes</code>), la fenêtre utilisée	le tableau de bord, à chaque rafraîchissement
<code>GET /pilotage/journal? depuis=&tag=&evenement=& limite=</code>	lecture paginée de <code>ops/journal.jsonl</code>	le tableau de bord ; un script d'audit
<code>POST /pilotage/rollback</code>	déclenche <code>redeploy(tag_cible)</code> ; corps { <code>tag_cible</code> , <code>acteur</code> , <code>commentaire</code> }	le bouton du tableau de bord — et rien d'autre en pratique, mais l'API existe indépendamment de lui
<code>GET /pilotage/seuils</code>	lecture de <code>thresholds.yml</code> en JSON, pour audit	le tableau de bord, un développeur

`POST /pilotage/rollback` répond `201` avec la ligne de journal créée. Erreurs : `422` si `acteur` est absent (le clic doit avoir un nom, jamais anonyme) ; `409` s'il n'existe aucun tag précédent validé à redéployer. Accès restreint à un jeton d'administration, distinct de la clé cliente de `/v2` — cette API contrôle la production, elle n'analyse pas de contrats.

Ce qui n'est délibérément pas une route API : l'écriture des seuils. `thresholds.yml` ne se modifie que par une pull request (section 9) ; une route `POST /pilotage/seuils` court-circuiterait la trace par commit que cette procédure garantit. L'API de pilotage agit sur la production ; elle ne réécrit jamais sa propre configuration.

7. Traces et collecteur

Question : à quoi servent les traces si elles ne pilotent rien ? **Réponse courte :** au diagnostic d'une requête précise après une alerte ; OTLP imposé, Jaeger choisi, Langfuse nommé.

Le protocole est imposé : **OTLP** (OpenTelemetry). Les spans existent déjà (`analyse.requete` → `llm.appel` , avec version, troncature, latence, tokens — téléométrie fournie) ; il manque l'exporteur OTLP et un collecteur.

Choix : Jaeger, en un conteneur « all-in-one » dans le Docker Compose — OTLP natif, une interface où l'on retrouve une requête par son `request_id`, aucune base à administrer. **Alternative nommée : Langfuse** — orienté LLM (prompts, coûts et scores par appel, jeux de données, alertes dans son offre Cloud). Écarté ici parce que l'auto-hébergement est lourd pour une personne, et que nos alertes viennent du watcher, pas du

collecteur. Le formateur a imposé le protocole, pas l'outil : changer de collecteur ne change ni la production ni le pilotage.

Rôle des traces dans ce chantier : **diagnostic**, pas pilotage. Quand une alerte tombe, la trace de la requête en cause dit où le temps est passé et ce que le fournisseur a répondu ; c'est la matière du Chantier « résolution d'incident » (C21), pas un signal de décision.

8. Le tableau de bord

Question : que montre le tableau de bord, et d'où ? **Réponse courte** : les cinq signaux par tag depuis `metrics.jsonl`, l'état du watcher, le journal et le bouton — en étendant le tableau fourni, pas en le remplaçant.

Le fourni (`ops/dashboard.py`) lit `metrics.jsonl` et le registre [**à confirmer : technologie et périmètre exact**]. On l'étend, on ne le remplace pas :

- les cinq signaux par tag, dont la **distribution** du score en déciles et la part $< 0,5$ — pas une moyenne ;
- la part de trafic $v1/v2$ et, dans $/v2$, la part par tag — comparée au poids attendu ;
- l'état du watcher par tag (ok · alerte · données insuffisantes) et la bannière ;
- les vingt dernières lignes du journal ;
- le bouton de rollback (section 6) ;
- la version qui sert chaque part de trafic, lue dans le registre — la preuve du test d'acceptance 4, vue de la production.

Test d'acceptance 9 : ouvrir le tableau de bord et voir latence, coût, erreurs, distribution du score, part $v1/v2$.

9. Ajuster les seuils

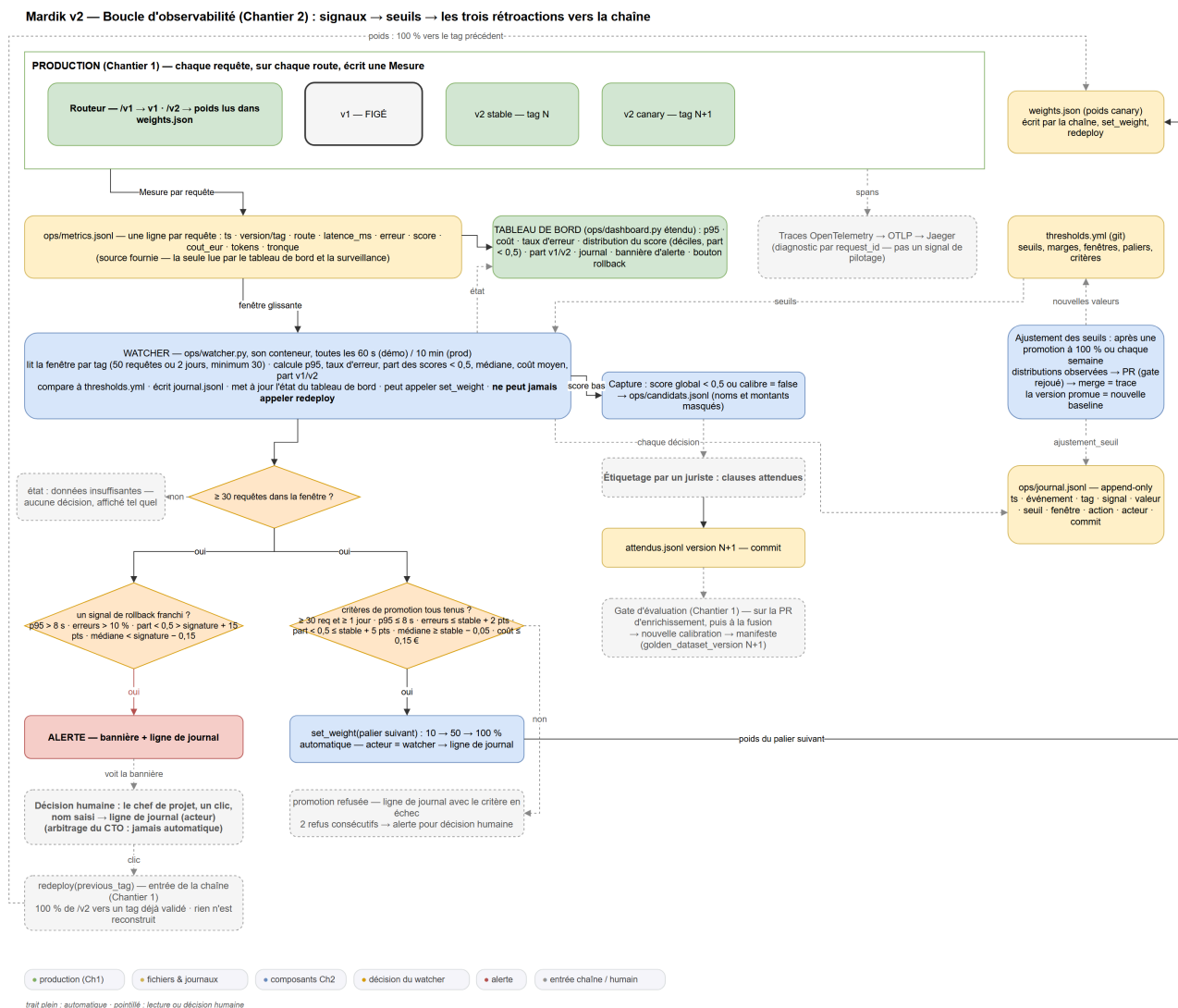
Question : comment un seuil change-t-il sans « doigt mouillé » ? **Réponse courte** : mesurer, proposer, une pull request, une ligne de journal avec le commit — et la version promue devient la baseline.

Une procédure, pas une habitude :

1. **Quand** : après chaque promotion à 100 %, ou chaque semaine.
2. **Mesurer** : sur la période, les distributions réelles des cinq signaux par tag (le watcher les exporte en une table).
3. **Proposer** : de nouvelles valeurs de `thresholds.yml` — les marges d'abord, jamais le principe.
4. **Tracer** : une pull request qui modifie `thresholds.yml` — elle déclenche le gate d'évaluation (filtre de chemin, Chantier 1 §3.0) : un seuil qui ferait échouer la baseline se voit avant la fusion — ; la fusion est la trace (auteur, diff, date) ; une ligne `ajustement_seuil` dans le journal cite les valeurs observées et le commit.
5. **Nouvelle baseline** : la version promue à 100 % devient `baseline` — sa signature remplace la précédente.
« Celle qui bat la baseline devient la baseline. »

Ce qu'on ne fait jamais : baisser un seuil pour faire passer une promotion. Si une promotion est refusée à répétition, on corrige la version, on n'assouplit pas la règle — même principe que le `xfail strict` du gate d'évaluation. Des seuils modifiables directement depuis le tableau de bord — vus dans la conception d'un camarade le 18/09 — ont été écartés : un champ de saisie casse la trace par commit que cette procédure garantit.

10. Schéma de la boucle d'observabilité



Source éditable : [schemas/schema-boucle-observabilite.drawio](#).

Comment le lire. En haut, en vert, la production du Chantier 1 : chaque requête écrit une *Mesure*. La colonne de droite, en jaune, ce qui est versionné : `weights.json`, `thresholds.yml`, le journal — et, en bleu, la procédure d'ajustement. Au centre, le watcher lit la fenêtre, compare aux seuils, et pose trois questions (losanges orange) : assez de données ? un signal de retour arrière franchi ? les critères de promotion tenus ? En bas à gauche, la boucle de retour arrière — rouge pour l'alerte, gris pointillé pour la décision humaine et l'entrée de la chaîne. Au centre, la promotion automatique. À droite, l'enrichissement : capture, étiquetage, nouvelle version du jeu de référence par pull request, gate d'évaluation rejoué sur cette PR puis à la fusion. Les traits pleins sont automatiques ; les pointillés sont des lectures ou des décisions humaines.

11. Tableau de pilotage

Le livrable demandé par le brief : pour chaque signal, le seuil, la rétroaction déclenchée et la trace laissée. Les valeurs sont celles de `thresholds.yml` (valeurs de travail, section 3).

Signal	Calcul et fenêtre	Seuil	Rétroaction déclenchée	Trace (ligne du journal)
Latence p95	p95 de <code>latence_ms</code> par tag ; 50 requêtes ou 2 jours, min 30	> 8 000 ms	alerte → décision humaine (rollback) ; refus de promotion	<code>alerte</code> · <code>signal=p95_ms</code> · valeur · seuil · fenêtre · tag
Taux d'erreur	part des <code>erreur=true</code> ; même fenêtre	> 10 % (alerte) · > stable + 2 pts (promotion)	alerte → décision humaine ; refus de promotion	<code>alerte</code> / <code>refus_promotion</code> · <code>signal=taux_erreur</code>
Part des scores < 0,5	part des <code>score</code> < 0,5 ; même fenêtre	> signature + 15 pts (alerte) · > stable + 5 pts (promotion)	alerte → décision humaine ; refus de promotion	<code>signal=part_score_bas</code> · valeur · seuil (signature ou stable)
Médiane du score	médiane des <code>score</code> ; même fenêtre	< signature – 0,15 (alerte) · < stable – 0,05 (promotion)	alerte → décision humaine ; refus de promotion	<code>signal=mediane_score</code>
Coût moyen	moyenne de <code>cout_eur</code> ; même fenêtre	> 0,15 €	refus de promotion (pas de rollback) ; alerte coût si cumul du jour > budget/22	<code>refus_promotion</code> / <code>alerte</code> · <code>signal=cout_moyen_eur</code>
Part de trafic v1/v2	part des lignes par route et par tag ; par jour	part /v2 du canary ≠ poids attendu ± 5 pts	alerte rou tage → décision humaine	<code>alerte</code> · <code>signal=part_trafic</code> · valeur · poids attendu
Données insuffisantes	< 30 requêtes dans la fenêtre	—	aucune décision ; état affiché ; aucune promotion	<code>donnees_insuffisantes</code> · tag · nombre de requêtes
Critères de promotion tous tenus	les cinq, ≥ 30 requêtes et ≥ 1 jour au palier	voir <code>promotion</code>	<code>set_weight(palier suivant)</code> — automatique	<code>promotion</code> · acteur · watcher · palier · valeurs des cinq critères
Deux refus consécutifs	compteur par tag	2	alerte → décision humaine (attendre ou reculer)	<code>alerte</code> · <code>signal=refus_consecutifs</code>
Alerte suivie d'un clic	—	—	<code>redeploy(previous_tag)</code> — humain	<code>rollback</code> · acteur = nom saisi · tag quitté · tag repris · alerte d'origine
Cas à faible confiance	<code>score</code> < 0,5 ou <code>calibre=false</code> , par requête	—	capture → étiquetage → <code>attendus.jsonl</code> N+1 → gate suivant	<code>capture</code> (<code>request_id</code> , tag) puis <code>enrichissement</code> (<code>dataset_version</code> , <code>commit</code>)
Distributions observées	table de la période	après promotion à 100 % / hebdo	PR sur <code>thresholds.yml</code> ; nouvelle baseline	<code>ajustement_seuil</code> · valeurs observées · <code>commit</code>

12. Démonstration et tests d'acceptance

Chaque test est une commande et une observation, pas une affirmation. Les commandes `make` sont celles du dépôt fourni [à confirmer].

Test	Ce qu'on lance	Ce qu'on observe
6 — dérive simulée → rollback via la chaîne, tracé	<code>make traffic MODE=derive-score</code> avec <code>DRIFT=score</code> (le proxy effondre les scores vers 0,5) ; watcher en cadence démo	la part < 0,5 franchit signature + 15 pts → bannière + ligne alerte ; clic → <code>weights.json</code> à 100 % sur le tag précédent → ligne <code>rollback</code> avec l'acteur ; le tableau de bord montre la part de trafic du tag précédent

Test	Ce qu'on lance	Ce qu'on observe
7 — promotion si conforme, pas de promotion sinon	trafic sain ; <code>requetes_min</code> abaissé en configuration de démo ; puis <code>DRIFT=erreurs</code>	premier cas : ligne <code>promotion</code> , poids 10 → 50 ; second cas : ligne <code>refus_promotion</code> avec <code>taux_erreur</code> , poids inchangé
8 — cas à faible confiance → jeu d'éval → rejoué à la fusion suivante	une requête à score bas ; étiquetage d'un candidat ; script d'ajout ; fusion sur <code>main</code>	<code>candidats.jsonl</code> contient le cas ; <code>attendus.jsonl</code> passe en N+1 (commit) ; le journal de la chaîne montre le gate jouant N+1 ; le manifeste de la release porte <code>golden_dataset_version: N+1</code>
9 — tableau de bord	ouvrir le tableau de bord	latence, coût, erreurs, distribution du score (déciles + part < 0,5), part v1/v2 — par tag
10 — journal de pilotage	ouvrir <code>ops/journal.jsonl</code> ou le panneau du tableau de bord	chaque ajustement (seuil, promotion, rollback) avec le signal qui l'a déclenché, l'acteur, la fenêtre

Avant la démonstration, la mesure du détecteur dans les deux sens (section 3) est jouée et ses deux chiffres notés. Preuve d'exécution (runbook, § 7) : une transcription du test 6 de bout en bout — dérive lancée, détection, rollback, journal.

12 bis. Choix d'outils non triviaux — ce qui est retenu, ce qui est écarté, pourquoi

Les outils fournis par la remédiation (télémétrie, tableau de bord, proxy de dérive) et les outils familiers ne sont pas justifiés ici ; ceux qui suivent le sont, parce qu'une alternative existait.

Retenu	Alternative écartée	Pourquoi	Section
Jaeger all-in-one comme collecteur	Langfuse auto-hébergé	OTLP natif, un conteneur, aucune base à administrer ; les alertes viennent du watcher, pas du collecteur ; Langfuse Cloud reste l'alternative nommée (H26)	§7
Watcher = processus dans son propre conteneur	tâche planifiée du serveur d'intégration (<code>schedule</code> GitHub Actions)	un exécuteur distant ne voit pas une production locale ; le watcher vit là où sont les mesures	§6.1
<code>metrics.jsonl</code> + calculs par fenêtre	base de séries temporelles (Prometheus, InfluxDB)	fichier fourni par la remédiation, déjà lu par le tableau de bord ; ~400 lignes par mois se lisent directement ; une base ajouterait un composant sans ajouter un signal	§2
Seuils git-only (<code>thresholds.yml</code> par pull request)	seuils modifiables depuis le tableau de bord	la trace par commit (auteur, diff, date) ; l'option « tableau de bord », vue dans une conception de camarade, casserait cette trace	§9
API de pilotage (<code>/pilotage/*</code>)	bouton seul, dans le code du tableau de bord	un contrat vérifiable et rejouable par un script, même discipline que <code>/v2</code> ; l'écriture des seuils en est délibérément exclue	§6.4
Masquage par regex avant capture	anonymisation outillée (reconnaissance d'entités)	partiel par construction et dit tel quel ; la question reste ouverte avec le client (H24) — un outil de plus ne la fermerait pas sans son accord	§4.3
Fenêtre « 50 requêtes ou 2 jours »	fenêtre en requêtes seules	à ~20 requêtes par jour, une fenêtre comptée en requêtes seules peut durer des semaines ; la borne en jours garde la décision datée	§3

13. Hypothèses, limites, à confirmer

Hypothèses (H) et ce qui se passe si elles sont fausses

#	Hypothèse	Si elle est fausse
H20	Les marges de <code>thresholds.yml</code> (+15 pts, -0,15, +5 pts, -0,05, 10 %) sont des valeurs de travail	posées sur les premières distributions réelles, par la procédure de la section 9 — c'est prévu
H21	30 requêtes suffisent pour qu'une part et une médiane soient stables	si les fausses alertes sont trop fréquentes, le minimum monte ; la mécanique ne change pas
H22	<code>deploy.py::surveiller</code> sait lire une fenêtre par tag et calculer des agrégats	sinon le watcher lit <code>metrics.jsonl</code> directement — quelques lignes de plus, même contrat
H23	Le tableau de bord fourni est extensible (page web)	sinon le bouton et la bannière vivent sur la page d'administration du frontend v2
H24	Le masquage par regex suffit pour stocker des candidats	sinon : stockage sans texte (seulement <code>request_id</code> et clauses), le juriste relit le contrat dans son outil
H25	Un juriste accepte d'étiqueter 2 à 4 cas par jour	sinon : étiquetage hebdomadaire, ou par le chef de projet ; le rythme de la boucle ralentit, pas sa forme
H26	Jaeger all-in-one tient en un conteneur à côté de la production	sinon Langfuse Cloud ; le protocole OTLP ne change pas
H27	Le canary reçoit assez de trafic en production (~2 requêtes/jour à 10 %) pour atteindre 30 requêtes en un temps raisonnable	à 10 % de 20/jour, 30 requêtes = 15 jours : trop long. Réponse prévue : premier palier à 25 % plutôt que 10 %, décidé au vu du trafic réel ; ou fenêtre en jours avec un minimum plus bas au premier palier, dit dans <code>thresholds.yml</code>

Hors périmètre. Le retour arrière automatique (exclu par le CTO) · une alerte multi-canal (courriel/messagerie = option) · l'anonymisation robuste des contrats (question ouverte avec le client) · l'auto-hébergement de Langfuse · toute modification du contrat ou de la chaîne.

Limites et ce qui n'est pas fait.

- Rien n'est codé, rien n'est mesuré ; les seuils sont des valeurs de travail ; le détecteur n'a pas encore été mesuré dans les deux sens.
- Le dossier est écrit **avant la validation du Chantier 1** : il suppose confirmés le rollback manuel, `metrics.jsonl` comme source unique, et les deux entrées `set_weight` / `redeploy`. Si le formateur en infirme un, la section correspondante est reprise. Depuis le 18/09, il s'appuie aussi sur les révisions du Chantier 1 de ce jour : gate d'évaluation joué sur la pull request par filtre de chemin (dont `attendus.jsonl` et `thresholds.yml`), revue automatisée de PR, pas d'environnement de staging.
- **Divergences assumées avec une conception vue en classe le 18/09** : un rollback **automatique** sur seuil — exclu ici par l'arbitrage du CTO du 16/09, le watcher alerte et l'humain clique ; des seuils modifiables depuis le tableau de bord — écartés (section 9) ; une fenêtre sans borne en jours — la nôtre en a une, parce qu'à vingt requêtes par jour une fenêtre comptée en requêtes seules peut durer des semaines.
- Le code fourni n'a pas été lu (dépôt fermé) : toute phrase marquée **[à confirmer]** peut changer à l'ouverture ; la mécanique (fenêtre, seuils, trois boucles, journal) ne dépend pas de ces détails.
- Le volume de production (20 requêtes/jour) rend les fenêtres longues (H27) ; la démonstration se fait sur trafic simulé, et le dossier ne prétend pas que la démonstration est la production.
- Aucune revue externe avant le formateur.

14. Critères RNCP couverts

Compétence	Où dans ce dossier	Ce que le jury peut vérifier	Statut
C9 — Développer une API exposant un modèle d'IA (appliqué au pilotage)	§6.4	l'API de pilotage : contrat en schémas, erreurs explicites, ce qui est délibérément exclu (écriture des seuils)	conçu ; prouvé par <code>tests/test_pilotage*.py</code> au développement
C11 — Monitorer un modèle d'IA	§2, §8	les cinq signaux définis avec leur calcul et leur fenêtre, expliqués sans confusion (distribution $\neq$ moyenne, p95 $\neq$ moyenne) ; un vecteur de restitution temps réel (tableau de bord)	conçu ; prouvé par le tableau de bord en marche
C20 — Surveiller une application d'IA	§3, §6, §7, §11	un seuil et une valeur d'alerte par métrique à risque , tous dans <code>thresholds.yml</code> ; choix d'outillage argumentés (Jaeger vs Langfuse, watcher vs tâche planifiée) ; collecte, journalisation, restitution et alertes conçues pour fonctionner en local	conçu ; prouvé par le test 6
C21 — Résoudre les incidents techniques	§4.1, §5, §12	procédure de retour arrière en une commande, tracée avec sa cause ; traces par <code>request_id</code> pour le diagnostic ; preuve d'exécution transcrite	conçu ; prouvé par la transcription du test 6
C13 / C19 — chaîne et processus de livraison continue	§4.2, §9	promotion pilotée par les métriques comme étape de la livraison ; seuils versionnés, ajustés par pull request	conçu ; prouvé par les lignes <code>promotion</code> et un <code>commit</code> d'ajustement
C12 (non visée, touchée) — tests automatisés du modèle	§4.3	le jeu d'évaluation s'enrichit des cas réels et est rejoué par le gate	conçu
C4 · C5 · C8 (transversales)	§13, annexe A, §4.1	hypothèses et limites écrites ; réponses aux questions de cadrage ; l'arbitrage du CTO respecté et expliqué	conçu / en cours

Annexe A — Réponses aux questions de cadrage (Chantier 2)

Question (résumée)	Réponse en une ligne	Section
Quels signaux observer ? Pourquoi la distribution du score plutôt que sa moyenne ?	latence p95, coût, taux d'erreur, part des scores $< 0,5$ + médiane, part v1/v2 ; la moyenne cache un modèle sûr et faux sur les contrats longs	§2
Quel est l'état actuel de l'observabilité, et vers quoi aller ?	v1 émet déjà spans, journaux et <code>Mesure</code> (remédiation) ; v2 et le routeur émettent la même chose + le score ; le pilotage vient de <code>metrics.jsonl</code>	§1, §2
Comment rendre une requête traçable de bout en bout ; quels points instrumenter, quelles informations conserver ?	<code>request_id</code> sur chaque ligne et chaque span ; spans <code>analyse.requete</code> $\rightarrow$ <code>llm.appel</code> ; version, tag, configuration dans le manifeste et la <code>Mesure</code>	§7, Ch1 §4
Comment remonter et stocker les traces sans tout casser ? Quel client derrière OpenTelemetry — Langfuse, Jaeger ?	exporteur OTLP ajouté à la télémétrie existante ; Jaeger en un conteneur ; Langfuse nommé comme alternative	§7

Question (résumée)	Réponse en une ligne	Section
Comment distinguer une vraie dérive du bruit ?	fenêtre glissante, minimum 30 requêtes, marges dans <code>thresholds.yml</code> , référence = signature de la release ; détecteur mesuré dans les deux sens	\$3
Quels signaux en amont d'un rollback ? Automatique ou validé par un humain ? Comment passe-t-il par la chaîne ?	p95, taux d'erreur, part < 0,5, médiane ; humain , après alerte ; par <code>redeploy(previous_tag)</code> qui réécrit <code>weights.json</code>	\$4.1, \$11
Quels critères, pendant combien de temps, pour passer de 10 à 50 puis 100 % ? Que se passe-t-il si les critères ne sont pas tenus ?	cinq critères relatifs à la version stable, ≥ 30 requêtes et ≥ 1 jour par palier ; automatique si tous tiennent ; sinon refus tracé, deux refus → décision humaine	\$4.2, \$11
« What steers the migration ? » — les cinq signaux ou des critères métier ? Dérive par rapport à quoi ? Combien de requêtes ?	les cinq signaux, relatifs (stable vs canary) pour avancer, absolus (signature) pour alerter ; 30 requêtes minimum	\$3, \$4
Quelles informations sur le tableau de bord ? Quels seuils déclenchent une décision ?	\$8 pour le contenu ; tous les seuils en \$3 et \$11	\$8, \$11
Comment capturer les cas réels à faible confiance (anonymisation ?), comment entrent-ils dans le jeu d'éval, quand sont-ils rejoués ?	capture par règle, masquage regex (partiel, question ouverte), étiquetage par un juriste, version N+1 par pull request, gate rejoué sur cette PR puis à la fusion	\$4.3
Comment ajuster les seuils à partir des distributions observées, et tracer chaque ajustement ?	procédure : mesurer → proposer → PR sur <code>thresholds.yml</code> → ligne <code>ajustement_seuil</code> avec les valeurs et le commit ; la version promue devient la baseline	\$9
Qui regarde le tableau de bord, à quelle fréquence, par quel canal est-il alerté ?	le chef de projet ; bannière + journal, courriel en option ; le watcher regarde, lui, toutes les 10 minutes	\$6.1, \$6.2
Le rollback est-il seulement un bouton, ou un contrat vérifiable ?	une API de pilotage (<code>/pilotage/rollback</code> , <code>/pilotage/journal</code> , <code>/pilotage/etat</code> , <code>/pilotage/seuils</code>), même discipline que <code>/v2</code> ; les seuils restent en écriture git-only	\$6.4

Glossaire

- **Boucle de rétroaction** : un mécanisme où une mesure de la production déclenche une action sur la chaîne de livraison.
- **Signal** : une grandeur calculée à partir des mesures par requête, sur une fenêtre, par version.
- **Fenêtre glissante** : le sous-ensemble des dernières mesures (ici 50 requêtes ou 2 jours) sur lequel un signal est calculé.
- **Échantillon minimal** : le nombre de mesures (30) en dessous duquel aucun signal n'est jugé fiable.
- **p95** : la valeur sous laquelle 95 % des requêtes se situent.
- **Médiane** : la valeur qui sépare la moitié haute de la moitié basse — insensible aux extrêmes.
- **Part des scores < 0,5** : la proportion de réponses dont le score global est dans la bande « basse » — ce que le juriste ressent comme « je dois relire ».
- **Signature de release** : la distribution du score obtenue par une version sur les 12 contrats de référence au moment du gate ; conservée dans le manifeste.
- **Dérive** : l'écart entre la distribution observée en production et la référence (signature, ou version stable).
- **Watcher** : le processus qui lit les mesures, applique les seuils, alerte et promeut — jamais ne recule.
- **Alerte** : un état et une ligne de journal qui appellent une décision humaine.
- **Promotion** : passage du canary au palier suivant (10 → 50 → 100 %) par écriture du poids.
- **Retour arrière (rollback)** : redéploiement d'un tag précédent déjà validé, par `redeploy(previous_tag)`.
- **Critères non compensables** : chacun doit tenir ; un bon résultat sur l'un ne rachète pas un échec sur l'autre.

- **Baseline** : la version de référence que la suivante doit battre ; la version promue à 100 % devient la nouvelle baseline.
- **Journal de pilotage** : le fichier append-only des décisions (`ops/journal.jsonl`).
- **Capture** : la copie d'une requête à faible confiance dans les candidats à l'étiquetage.
- **Étiquetage** : écrire, pour un cas capturé, la réponse attendue — ici la liste des clauses.
- **Masquage** : remplacement des noms et montants par des jetons avant stockage ; partiel s'il est fait par regex.
- **OTLP** : le protocole de transport d'OpenTelemetry ; les traces partent dans ce format quel que soit le collecteur.
- **Collecteur** : le service qui reçoit les traces (Jaeger, Langfuse...).
- **Span** : un segment de trace correspondant à une étape d'une requête.
- **Fail-closed** : en l'absence de données fiables, ne rien autoriser (pas de promotion) plutôt que supposer que tout va bien.
- **Faux positif / taux de détection** : une alerte sans dérive réelle / la part des dérives réelles effectivement détectées — les deux se mesurent ensemble.
- `thresholds.yml` : le fichier versionné des seuils, marges, fenêtres et paliers.
- `weights.json` : le fichier versionné des poids de routage du canary.
- **Proxy de dérive** : le composant fourni qui dégrade le trafic exprès (`DRIFT=latence|erreurs|score`) pour tester la détection.
- **API de pilotage** : l'interface documentée (`/pilotage/...`) par laquelle le tableau de bord — ou tout script — lit l'état, le journal, les seuils, et déclenche un retour arrière ; le contrat existe indépendamment du bouton qui l'appelle.